Министерство науки и высшего образования Российской Федерации

Федеральное государственное автономное образовательное
учреждение высшего образования
Национальный исследовательский Нижегородский государственный
университет им. Н.И. Лобачевского

Институт информационных технологий, математики и механики

Отчет по лабораторной работе №1

«Множества на основе битовых полей»

Выполнил:
студент группы ФИсп1
Плакунов Ф. П.

Проверил:
Ивлев А.Д.

Нижний Новгород
2026

Содержание

Введение 3
1. Постановка задачи 4
2. Руководство пользователя 5
3. Руководство программиста 6
3.1. Описание структуры программы 6
3.2. Описание структур данных 6
3.3. Описание алгоритмов 6
4. Результаты экспериментов и тестирования 8
Заключение 9
Литература 10

Введение

В данной лабораторной работе мы изучаем основы объектно-ориентированного программирования на языке C++ и учимся использовать фреймворк Google Test. В качестве задачи нам нужно реализовать два собственных класса: битовое поле (TBitField) и множество (TSet), которые должны эффективно использовать память. Также в работе требуется настроить юнит-тестирование с помощью фреймворка Google Test, чтобы проверить, что наш код работает без ошибок, как в обычных ситуациях, так и в краевых случаях (например, при выходе за границы массива).

1. Постановка задачи

Цель работы: разработать классы TBitField и TSet, а также написать для них тесты. Для TBitField нужно реализовать:

- Хранение битов в динамическом массиве.
- Методы установки (SetBit), очистки (ClrBit) и получения (GetBit) бита.
- Перегрузку операторов ==, !=, =, |, &, ~, а также ввод/вывод.
- Обработку исключений, если индекс выходит за пределы длины поля.

Для TSet (который использует внутри себя TBitField) нужно реализовать:

- Добавление (InsElem), удаление (DelElem) и проверку наличия (IsMember) элемента.
- Операции объединения (+), пересечения (*), разности (-), дополнения (~) и работу с отдельными элементами.
- Ввод и вывод элементов множества должен быть в виде номеров элементов через пробел (например: 2 5 9), а не в виде нулей и единиц.
- Исключения для неверных размеров и выхода за границы.

2. Руководство пользователя

Программа представляет собой консольное приложение (хотя основной упор сделан на библиотеку классов) и тестовый проект.
При работе с множеством (TSet) пользователь может вводить элементы через пробел. При вводе программа считывает числа и добавляет их в множество. Если ввести число, которое больше максимальной мощности множества или меньше нуля, программа выдаст ошибку (исключение).
При выводе множества на экран, программа не выводит длинную строку из нулей и единиц, как обычное битовое поле, а печатает только те номера, которые реально присутствуют в множестве, разделяя их пробелом. Это очень удобно для восприятия человеком.
Ниже на рисунках 1 и 2 показано, как программно осуществляется ввод и вывод данных с использованием потоков.

```
std::stringstream ss;
ss << bf;
EXPECT_EQ(ss.str(), "10101");
TBitField bf2(5);
std::stringstream ss_in("01010");
ss_in >> bf2;
```

Рис. 1. Пример ввода и вывода для TBitField

```
std::stringstream ss;
ss << set;
EXPECT_EQ(ss.str(), "2 5 9");
TSet set_in(20);
std::stringstream ss_in("1 4 7");
ss_in >> set_in;
```

Рис. 2. Пример ввода и вывода для TSet

3. Руководство программиста

3.1. Описание структуры программы

В программе реализованы два основных пользовательских класса: TBitField (битовое поле) и TSet (множество).

Класс TBitField представляет собой динамический массив битов. Его внутренняя структура включает три поля:

- BitLen (тип int) — логическая длина поля в битах (количество битов, с которыми может работать пользователь).
- MemLen (тип int) — физическая длина массива памяти (количество ячеек типа TELEM, необходимых для хранения BitLen битов).
- pMem (тип TELEM*) — указатель на динамически выделенный массив памяти. TELEM — это псевдоним для целочисленного типа (unsigned int).

Память выделяется в конструкторе с помощью оператора new[] и освобождается в деструкторе через delete[]. Если создается пустое поле (BitLen == 0), память не выделяется, а указатель pMem инициализируется как nullptr. Класс владеет ресурсом, поэтому в нем реализованы конструктор копирования и перегружен оператор присваивания (правило трех), обеспечивающие глубокое копирование.

Класс TSet представляет собой математическое множество. В данной работе используется композиция, а не наследование, так как множество и битовое поле — разные математические объекты. Внутри класса TSet содержатся:

- MaxPower (тип int) — мощность универсума (максимально возможное количество элементов в множестве).
- BitField (тип TBitField) — объект битового поля, который выступает в роли характеристического вектора. Если элемент с индексом i присутствует во множестве, то i-й бит в BitField установлен в 1.

Оценка памяти: Оба класса используют память крайне эффективно. Для хранения NN битов или элементов множества расходуется O(N)O(N) памяти, при этом один байт хранит информацию о 8 элементах.

3.2. Описание структур данных

В программе реализованы два основных пользовательских класса: TBitField (битовое поле) и TSet (множество).

Класс TBitField представляет собой динамический массив битов. Его внутренняя структура включает три поля:

- BitLen (тип int) — логическая длина поля в битах (количество битов, с которыми может работать пользователь).
- MemLen (тип int) — физическая длина массива памяти (количество ячеек типа TELEM, необходимых для хранения BitLen битов).

- pMem (тип TELEM*) — указатель на динамически выделенный массив памяти. TELEM — это псевдоним для целочисленного типа (unsigned int).

Память выделяется в конструкторе с помощью оператора new[] и освобождается в деструкторе через delete[]. Если создается пустое поле (BitLen == 0), память не выделяется, а указатель pMem инициализируется как nullptr. Класс владеет ресурсом, поэтому в нем реализованы конструктор копирования и перегружен оператор присваивания (правило трех), обеспечивающие глубокое копирование.

Класс TSet представляет собой математическое множество. В данной работе используется композиция, а не наследование, так как множество и битовое поле — разные математические объекты. Внутри класса TSet содержатся:

- MaxPower (тип int) — мощность универсума (максимально возможное количество элементов в множестве).

- BitField (тип TBitField) — объект битового поля, который выступает в роли характеристического вектора. Если элемент с индексом i присутствует во множестве, то i-й бит в BitField установлен в 1.

Оценка памяти: Оба класса используют память крайне эффективно. Для хранения N битов или элементов множества расходуется O(N) памяти, при этом один байт хранит информацию о 8 элементах.

3.3. Описание алгоритмов

Алгоритмы в классе TBitField (Битовое поле):

- **Вычисление индекса и маски (Сложность: O(1)).** Память организована в виде массива ячеек (слов) TELEM. Для доступа к биту с номером n вычисляется индекс ячейки: n / (sizeof(TELEM) * 8), и позиция внутри ячейки: n % (sizeof(TELEM) * 8). Маска создается путем побитового сдвига единицы влево на вычисленную позицию.
- **Установка, сброс и чтение бита (Сложность: O(1)).** Установка (SetBit) выполняется через побитовое ИЛИ (|=) с маской. Сброс (ClrBit) выполняется через побитовое И (&=) с инвертированной маской. Чтение (GetBit) выполняется через побитовое И (&) ячейки и маски. Если результат не равен нулю, бит установлен.
- **Бинарные операции | (ИЛИ) и & (И) (Сложность: O(max(BitLen_1, BitLen_2))).** В отличие от стандартных массивов, операции допускают работу с полями разной длины. Алгоритм создает результирующее поле длиной, равной максимуму из двух исходных. Затем в цикле по всем ячейкам памяти применяет операцию к парам элементов. Если одна из строк короче, недостающие старшие ячейки считаются нулями (дополнение нулями). Использование побитовых операций над целыми словами (по 32 или 64 бита) дает ускорение в десятки раз по сравнению с поразрядной обработкой.
- **Инверсия ~ (Сложность: O(N), где N — количество ячеек памяти).** Алгоритм инвертирует все биты в массиве. Важный нюанс: если BitLen не кратно размеру ячейки, в последней ячейке образуются «мусорные» единицы, которые не входят в логическую длину поля. Чтобы они не ломали логику (например, оператор ==), применяется специальная маска, которая принудительно зануляет эти лишние биты в конце.
- **Оператор присваивания = (Сложность: O(N), где N — количество ячеек памяти).** Для предотвращения утечек памяти и проблем с самоприсваиванием алгоритм сначала проверяет, не являются ли объекты одним и тем же (this != &bf, сравнение по адресам). Если объекты разные, выделяется новый блок памяти,

копируются данные, и только после этого освобождается старый блок. Это гарантирует безопасность при возникновении исключений (Strong Exception Safety).

- **Ввод/вывод (Сложность: O(BitLen))**. Вывод идет посимвольно в цикле от 0 до BitLen. Ввод считывает символы '0' и '1' и вызывает соответствующие методы SetBit или ClrBit. При неверном символе генерируется исключение.

Алгоритмы в классе TSet (Множество):

- **Хранение и базовые операции (Сложность: O(1))**. Поскольку внутри TSet лежит TBitField, базовые операции InsElem, DelElem и IsMember сводятся к вызову соответствующих битовых операций за константное время. При этом проверяется, не выходит ли переданный индекс за границы MaxPower.
- **Объединение (+), пересечение (*) и дополнение (~) (Сложность: O(MaxPower) или O(M/32), где M — количество ячеек).** Операции над множествами делегируются битовым операциям |, & и ~ внутреннего поля. Для корректной работы пересечения и объединения проверяется равенство мощностей (MaxPower), так как операции над множествами из разных универсумов не имеют математического смысла. Дополнение автоматически заботится о том, чтобы не захватить несуществующие элементы за пределами MaxPower.
- **Ввод/вывод (Сложность: O(MaxPower)).** В отличие от битового поля, множество выводится в человекочитаемом виде. Алгоритм вывода идет циклом от 0 до MaxPower, проверяет наличие элемента через IsMember и, если он есть, печатает его номер через пробел. Ввод работает аналогично: программа считывает целые числа из потока и добавляет их через InsElem, пока поток не закончится.
- **Обработка ошибок (Сложность: O(1)).** Во всех методах, принимающих индекс элемента, реализована проверка if (Elem < 0 || Elem >= MaxPower). При выходе за границы выбрасывается исключение std::invalid_argument, что защищает программу от segmentation fault.

4. Результаты экспериментов и тестирования

Для проверки работоспособности программы мы использовали фреймворк Google Test и написали более 20 тестов, разбитых на две группы (для TBitField и TSet).

Какие тесты были написаны:

1. **Базовые операции:** создание объектов, установка/сброс/чтение битов, добавление и удаление элементов множества.
2. **Краевые случаи и исключения:** проверка того, что при создании объекта с отрицательным размером бросается исключение. Проверка выхода за границы при установке бита или добавлении элемента (программа не должна падать с ошибкой сегментации, а должна выдать std::invalid_argument).
3. **Конструкторы и присваивание:** проверка конструктора копирования и оператора =. Важно, чтобы при присваивании объектов разного размера память перевыделялась корректно, без утечек.
4. **Логические операции:** проверка | и & для полей, + и * для множеств. Проверка того, что объединение множеств содержит элементы из обоих, а пересечение — только общие.
5. **Инверсия:** тест на то, что двойная инверсия (~~bf) возвращает исходный объект. Это косвенно доказывает, что алгоритм зануления «хвоста» работает правильно.
6. **Ввод/вывод:** с использованием std::stringstream мы проверяли, что битовое поле выводится как строка 0101..., а множество — как набор чисел через пробел (2 5 9). Проверялся и обратный процесс чтения из строки.
7. **Случайные данные:** стресс-тесты, где в цикле генерируются случайные индексы и проверяется, что установленные биты/элементы корректно считываются обратно.

Анализ результатов:

В ходе написания тестов были выявлены и исправлены ошибки. Например, изначально при инверсии поля «мусорные» биты в последней ячейке становились единицами, из-за чего тесты на равенство (==) падали. Также тесты показали, что старый вариант вывода множества выводил битовую строку, а не номера элементов. После исправлений все тесты успешно проходят (статус PASSED), что подтверждает корректность работы алгоритмов.

Заключение

В ходе выполнения лабораторной работы мы закрепили навыки работы с классами, динамической памятью, перегрузкой операторов и обработкой исключений. Также мы научились писать unit-тесты с помощью Google Test и обнаружили, что тесты реально помогают находить скрытые баги (например, проблему с неполным последним байтом при инверсии).

Литература

1. Страуструп Бьерн Язык программирования C++. Специальное издание. Пер. с англ. – М.: Издательство Бином, 2017 г. – 1136 с.
2. Сайт cppreference: https://en.cppreference.com/